Université Paris Cité, Campus Saint-Germain-des-Près
UFR de Mathématiques et Informatique
45, rue des Saints-Pères 75006 Paris

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Algorithmique et Programmation 2

Correction du partiel du 20 mars 2025 — durée: 1h30

Tout document interdit. Les calculatrices et les téléphones portables, même prévus à titre d'horloge, sont également interdits.

MERCI DE BIEN INDIQUER VOTRE GROUPE DE TD SUR VOTRE COPIE

Tous les exercices sont indépendants. On précisera bien la signature (type des entrées/sorties) de chaque fonction Python proposée. Ces signatures seront notées globalement sur 1 point.

Barème sur 21 points (en comptant le point global pour les signatures)

Exercice 1. **(au total sur 5 points)**

Pour chaque ligne (indépendante) d'instructions ci-dessous, donner le contenu de la variable `x` après exécution de la ligne considérée, ou expliquer le problème si l'instruction produit une erreur. **Expliquer votre réponse (une réponse correcte sans explication ne sera pas comptabilisée).**

```
1  x = bin(0b10)*(0b10+0b1)
2  x = int(3.2)+3/2+3//2
3  j = 1; x = abs(3+4j)
4  x = [[1],[2,3]]; y = x[:]; y[0].append(2)
5  L = list('python'); x = L.index('n')
6  x = set(range(4,1,-1)) & {3,0,1}
7  x = len( set([1,3]) - [1] )
8  x = {1,3} <= {3,1}
9  x = {1:2, 2:3, 3:4}.get(4,0)
10 L = lambda t: 1-t; x = L(L(3))
```

(1) `x = '0b100b100b10'` car `0b10` vaut 2 (spécifié en binaire), `bin(0b10)` vaut `'0b10'` (écriture binaire de 2), et `0b10+0b1` vaut 3 (2+1, spécifiés en binaire), donc `x` est obtenue en recopiant 3 fois la chaîne `'0b10'`.

(2) `int(3.2)` retourne 3 (arrondi sans partie décimale), `3/2` vaut 1.5 et `3//2` vaut 1 (quotient dans la division euclidienne), donc finalement `x = 5.5`.

(3) `x` est défini comme le module (fonction `abs` sur les complexes) du nombre complexe $3 + 4i$ (la définition `j=1` est inutile ici), donc $x = \sqrt{3^2 + 4^2} = 5$.

(4) `x` est une liste, `y` est une autre liste, mais qui contient les mêmes sous-listes que `x` puisqu'on n'a pas utilisé `deepcopy`. Par conséquent, l'ajout de 2 à la première sous-liste de `y` impacte aussi `x` et on a donc à la fin `x = [[1,2],[2,3]]`.

(5) On a `L = ['p','y','t','h','o','n']` donc `x` vaut 5 (indice de la première occurrence de la chaîne `'n'` dans la liste `L`).

(6) `x` est l'intersection (symbole `&`) de l'ensemble $\{4, 3, 2\}$ (résultat de `set(range(4,1,-1))` et de l'ensemble $\{3, 0, 1\}$, donc `x={3}`.

(7) L'expression `set([1,3]) - [1]` produit une erreur car l'opérateur `-` n'est pas défini entre un ensemble (set) et une liste.

(8) `x` est le résultat du test d'inclusion large de l'ensemble $\{1, 3\}$ dans lui-même, donc `x = True`.

(9) Cette instruction tente de calculer la valeur associé à la clé 4 dans le dictionnaire `{1:2, 2:3, 3:4}`. Comme 4 n'est pas une clé de ce dictionnaire, c'est la valeur par défaut (deuxième argument de la méhode `get()`) qui est retournée, c'est-à-dire `x = 0`.

(10) `x` est la valeur de $L(L(3))$, où L est la fonction définie par $L(t) = 1-t$. On a donc `x`$= 1-(1-3) = 3$.

Exercice 2. **(au total sur 2 points)**

On considère une fonction `f` définie avec l'en-tête suivant:

```
def f(x, y=1., z=False):
```

Pour chaque ligne ci-dessous, déterminer si l'appel à la fonction `f` est valide (dans ce cas, préciser la valeur de chaque argument dans `f`) ou si l'appel est invalide (dans ce cas, expliquer pourquoi).

```
1 f(z=True, 3, 2.)
2 f([])
3 f(z=True, x=2)
4 f(1, False, y=-2.)
```

(1) erreur, une spécification positionnelle (2e argument, 3) arrive après une spécification par mot-clé (`z=True`).
(2) appel correct, `x=[], y=1., z=False`
(3) appel correct, `x=2, y=1., z=True`
(4) erreur, l'argument `y` est spécifié deux fois, une fois de manière positionnelle (2e argument, `False`), et une fois par mot-clé (`y=-2.`).

Exercice 3. **(au total sur 2 points)**

Soit A une matrice carrée.

(1) **(sur 0,5 point)** Quelle est la complexité, en nombre de multiplications matricielles, du calcul de A^n par l'algorithme d'exponentiation rapide ?
La complexité est $O(\ln n)$ (voir cours).

(2) **(sur 1,5 point)** Expliquer comment calculer A^{45} avec cet algorithme. Combien de multiplications matricielles sont-elles nécessaires ?

On écrit successivement

$$\begin{aligned}
A^{45} &= A \times A^{44} \\
A^{44} &= A^{22} \times A^{22} \\
A^{22} &= A^{11} \times A^{11} \\
A^{11} &= A \times A^{10} \\
A^{10} &= A^{5} \times A^{5} \\
A^{5} &= A \times A^{4} \\
A^{4} &= A^{2} \times A^{2} \\
A^{2} &= A \times A
\end{aligned}$$

Le calcul effectif se fait en reprenant les lignes dans l'ordre inverse. Il y a au total 8 lignes, donc 8 multiplications matricielles à effectuer pour parvenir au résultat.

Exercice 4. **(au total sur 5 points)**

On considère une suite $(u_n)_{n\in\mathbb{N}}$ définie par $u_0 = -1$, $u_1 = 2$ et la relation de récurrence

$$\forall n \in \mathbb{N}, \quad u_{n+2} = \begin{cases} 2u_{n+1} + u_n & \text{si } u_n \leqslant 0, \\ u_{n+1} - 3u_n & \text{sinon.} \end{cases}$$

(1) **(sur 0,5 point)** Calculer u_5.

On a successivement:

$$\begin{aligned}
u_2 &= 2u_1 + u_0 = 4 - 1 = 3 && (n = 0, u_0 \leqslant 0) \\
u_3 &= u_2 - 3u_1 = 3 - 6 = -3 && (n = 1, u_1 > 0) \\
u_4 &= u_3 - 3u_2 = -3 - 9 = -12 && (n = 2, u_2 > 0) \\
u_5 &= 2u_4 + u_3 = -24 - 3 = -27 && (n = 3, u_3 \leqslant 0)
\end{aligned}$$

(2) **(sur 1,5 point)** Écrire une fonction **doublement récursive** suite(n) qui prend en entrée un entier naturel n et renvoie la valeur de u_n.

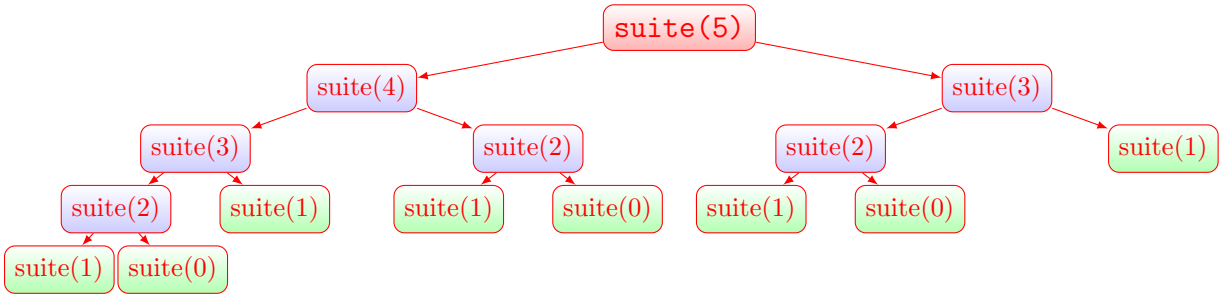
```
def suite(n):
    ''' int -> int
    retourne la valeur de u(n)
    '''
    if n==0:
        return -1
    if n==1:
        return 2
    u2 = suite(n-2) # u(n-2)
    u1 = suite(n-1) # u(n-1)
    if u2<=0:
        return 2*u1 + u2
    else:
        return u1 - 3*u2
```

Attention, l'implémentation ci-dessous n'est pas correcte, car elle effectue 3 appels récursifs au lieu de 2:

```
def suite(n):
    ''' int -> int
    retourne la valeur de u(n)
    '''
    if n==0:
        return -1
    if n==1:
```

```
        return 2
    if suite(n-2)<=0:
        return 2*suite(n-1) + suite(n-2)
    else:
        return suite(n-1) - 3*suite(n-2)
```

(3) **(sur 1,5 point)** Dessiner l'arbre des appels récursifs associés à l'appel `suite(5)`. Combien y a-t-il d'appels à la fonction `suite` ? De manière générale, quelle est la complexité, en nombre d'appels récursifs, de l'appel `suite(n)` ?



Il y a au total 15 appels à la fonction **suite** (c'est le nombre de nœuds de l'arbre).

De manière générale, nous avons vu en cours que la complexité d'une telle fonction doublement récursive est exponentielle (plus précisément, $O(\varphi^n)$, où $\varphi = \frac{\sqrt{5}+1}{2}$ est le nombre d'or).

(4) **(sur 1,5 point)** Écrire une fonction `suite2`, version modifiée avec mémoïsation de la fonction `suite`. Quel est l'intérêt de cette modification ?

```
D = {0:-1, 1:2} # dictionnaire pour la mémoisation

def suite2(n):
    ''' int -> int
    retourne la valeur de u(n)
    '''
    if n not in D:
        u2 = suite(n-2) # u(n-2)
        u1 = suite(n-1) # u(n-1)
        if u2<=0:
            D[n] = 2*u1 + u2
        else:
            D[n] = u1 - 3*u2
    return D[n]
```

L'intérêt de cette modification est qu'elle accélère énormément le calcul, puisque l'on passe d'une complexité exponentielle $O(\varphi^n)$ à une complexité linéaire $O(n)$ (avec la mémoïsation on ne calcule qu'une fois chaque u_k pour $k \leqslant n$).

Exercice 5. **(au total sur 6 points)**

(1) **(sur 1,5 point)**

Écrire une fonction `encode(s, D)`, qui prend en entrée une chaîne de caractères `s` et un dictionnaire `D` (dont les clés sont des chaînes de caractères de longueur 1 et les valeurs des chaînes de caractères quelconques), et retourne une chaîne `t` qui correspond à une copie de `s` dans laquelle les caractères qui sont des clés de `D` sont remplacés par les valeurs correspondantes dans `D`. Par exemple, si `s="Hello, world!"` et `D={'e':'3', 'l':'1', 'o':'00', ',':''}`, alors `encode(s, D)` doit retourner la chaîne de caractères `"H31100 w00r1d!"`

```
def encode(s, D):
    ''' str x dict -> str
    retourne l'encodage de s par D
    '''
```

```
    t = ""
    for c in s:
        if c in D:
            t += D[c]
        else:
            t += c
    return t
```

On peut simplifier le code précédent en utilisant ma méthode `get()` du type `dict`:

```
def encode(s, D):
    ''' str x dict -> str
    retourne l'encodage de s par D
    '''
    t = ""
    for c in s:
        t += D.get(c,c)
    return t
```

(2) **(sur 1,5 point)**

Écrire une fonction génératrice `pairs(L)`, qui prend en entrée une liste de nombres (entiers ou réels) `L` et délivre successivement, dans l'ordre, tous les éléments de `L` qui sont pairs. Par exemple, si `L = [1, 0, 2., 4, -1, -6]`, alors `pairs(L)` doit délivrer successivement les valeurs 0, 2., 4, et -6.

```
def pairs(L):
    ''' list -> generator
    délivre les nombres pairs de L
    '''
    for x in L:
        if x%2==0:
            yield x
```

(3) **(sur 1,5 point)**

Écrire une fonction `intersection(D1, D2)` qui prend en entrée deux dictionnaires `D1` et `D2`, et retourne un dictionnaire `D3`, dont les clés sont les clés communes à `D1` et `D2`, et la valeur correspondant à une clé `x` de `D3` est un tuple à 2 éléments, formé des valeurs associées à la clé `x` dans `D1` et `D2`. Par exemple, si `D1={1:2, 2:3, 3:4}` et `D2={2:0, 3:True, 4:1}`, alors `intersection(D1, D2)` doit retourner le dictionnaire `D3={2:(3,0), 3:(4,True)}`.

```
def intersection(D1, D2):
    ''' dict x dict -> dict
    retourne "l'intersection" des dictionnaires D1 et D2
    '''
    D = dict()
    for x in D1:
        if x in D2:
            D[x] = (D1[x], D2[x])
    return D
```

(4) **(sur 1,5 point)**

Écrire une fonction récursive `applique(f, L)`, qui prend en entrée une fonction `f` et une liste `L` (composée d'entiers et éventuellement, récursivement, de listes), et retourne une copie de la liste `L` dans laquelle toutes les valeurs entières ont été remplacées par leur image par la fonction `f`. Par exemple, si `f = lambda x:x+1` et `L = [1,[[2,3],0,[[1],[-1]]]]`, alors `applique(f, L)` doit retourner la liste `L = [2,[[3,4],1,[[2],[0]]]]`.

On construit la liste résultat (notons la `R` par exempl) élément par élément, à partir de chaque élément `x` de `L`:

- si `x` est un entier, on ajoute `f(x)` à `R`;

- sinon, c'est que `x` est une liste, et on ajoute à `R` sa "traduction" obtenue par l'appel récursif `applique(f, x)`.

On obtient ainsi le code suivant:

```
def applique(f, L):
    ''' function x list -> list
    applique une fonction récursivement sur les entiers d'une liste
    '''
    R = []
    for x in L:
        if type(x) is int:
            R.append(f(x))
        else:
            R.append(applique(f,x))
    return R
```

On peut si on le souhaite intégrer tout le code en une seule compréhension de liste:

```
def applique(f, L):
    ''' function x list -> list '''
    return [f(x) if type(x) is int else applique(f,x) for x in L]
```

À noter que la syntaxe `type(x)==int` est également correcte.